
深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 图论——桥问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2026年6月3日—2026年6月4日

实验报告提交时间： 2026年6月 日

教务部制

一、实验目的：

掌握图的连通性。

掌握并查集的基本原理和应用。

二、实验内容：

1. 桥的定义

在图论中，一条边被称为“桥”代表这条边一旦被删除，这张图的连通块数量会增加。等价地说，一条边是一座桥当且仅当这条边不在任何环上。一张图可以有零或多座桥。

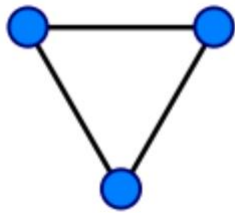


图 1 没有桥的无向连通图
(桥以红色线段标示)

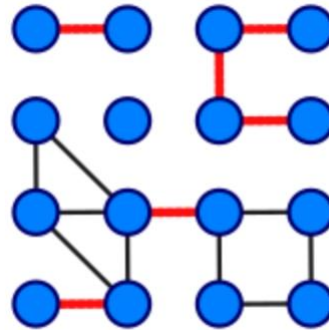


图 2 这是有 16 个顶点和 6 个桥的图

2. 求解问题

找出一个无向图中所有的桥。

3. 算法

(1) 基准算法

For every edge (u, v) , do following

a) Remove (u, v) from graph

b) See if the graph remains connected (We can either use BFS or DFS)

c) Add (u, v) back to the graph.

(2) 应用并查集设计一个比基准算法更高效的算法。不要使用 Tarjan 算法，如果使用 Tarjan 算法，仍然需要利用并查集设计一个比基准算法更高效的算法。

三、实验要求

1. 实现上述基准算法。
2. 设计的高效算法中必须使用并查集，如有需要，可以配合使用其他任何数据结构。
3. 用图 2 的例子验证算法正确性。
4. 使用文件 mediumG.txt 和 largeG.txt 中的无向图测试基准算法和高效算法的性能，记录两个算法的运行时间。
5. 设计的高效算法的运行时间作为评分标准之一。

四、实验内容及过程：

(一) 蛮力法

蛮力法过程：

依次枚举图中的每一条边，将该边暂时从图中删除，然后利用深度优先搜索（DFS）或广度优先搜索（BFS）判断图的连通性。如果删除该边后图不再连通，则说明该边是桥；否则该边不是桥。

具体步骤如下：

- ✧ 读入图的数据，建立邻接表存储结构。
- ✧ 依次遍历图中的每一条边 $e=(u,v)$ 。
- ✧ 暂时忽略边 e 。
- ✧ 从任意一个顶点开始执行 DFS（或 BFS），统计能够访问到的顶点数量。
- ✧ 若访问顶点数小于图中的总顶点数，说明删除边 e 后图被分割成多个连通分量，则边 e 为桥。
- ✧ 恢复边 e ，继续检测下一条边。
- ✧ 所有边检测完成后，输出全部桥边。

伪代码：

```
for each edge e in G
{
    temporarily remove e
    DFS(start_vertex)
    if visited_vertex_count < V e is a bridge
    restore e
}
```

时间复杂度分析：

我们假设图里有 E 条边。而对于图中的每一条边，都需要执行一次 DFS（或 BFS）来判断图的连通性。每次访问都会把点和边都访问一次，所以单次 DFS/BFS 的时间复杂度为 $O(V + E)$ ，因此总时间复杂度为：

$$O(E \times (V + E))$$

(二) 并查集

将每个顶点看作一个独立集合，即初始化父节点数组 $fa[i] = i$ 。随后依次读取图中的每一条边 (u, v) 。

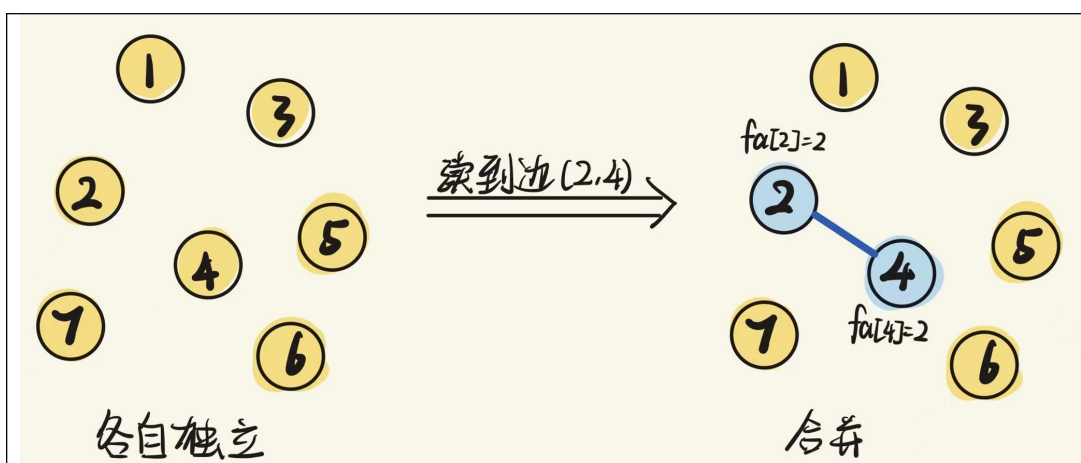


图 3 并查集生成森林示意图

对于当前边，首先分别查找顶点 u 和顶点 v 所在集合的根节点。若两者根节点不同，则说明两个顶点尚未连通，此时将两个集合进行合并；若两者代表节点相同，则说明两个顶点已经存在连接路径，当前边的加入将形成一个环。

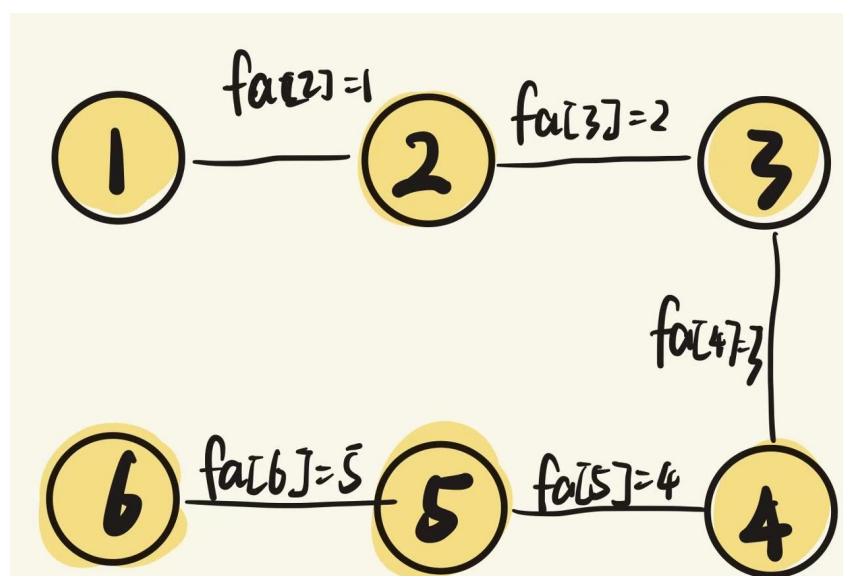


图 4 并查集合并与环检测示意图

并查集主要由父节点数组、查找操作（Find）和合并操作（Union）组成。其中查找操作用于寻找某个顶点所在集合的根节点，合并操作用于将两个不同集合连接成一个集合。

当并查集判断某条边 (u, v) 的两个端点已经属于同一个集合时，说明在当前生成森林中， u 到 v 之间已经存在一条路径。因此，边 (u, v) 与森林中 u 到 v 的路径共同构成一个环。

根据桥的定义，在环上的边一定不是桥。所以此时需要将该路径上的所有树边标记为“非桥边”，这个过程称为环覆盖。

具体实现时，需要在构造生成森林的过程中保存树结构信息，包括每个结点的父节点 `parent[]`、深度 `depth[]`，以及该结点与父节点之间对应的边编号 `parentEdge[]`。当遇到非树边 (u, v) 时，从深度较大的结点开始不断向上回溯，直到两个结点汇合。在回溯过程中，经过的每一条父边都被当前非树边所形成的环覆盖，因此将这些边标记为非桥边。

在不采用路径压缩优化的情况下，查找操作需要沿父节点链不断向上寻找根节点。

当集合结构退化为链状时，单次查找的时间复杂度可达到 $O(V)$ 。因此，对于包含 E 条边和 V 个顶点的图，总时间复杂度约为 $O(EV)$ 。

伪代码如下：

算法：并查集算法（无路径压缩）

输入：图 $G(V, E)$

输出：判断每条边加入时是否形成环

1. 初始化父节点数组 fa

2. 对于每个顶点 i ：

$fa[i] \leftarrow i$

3. 对于图中的每一条边 (u, v) ：

$rootU \leftarrow Find(u)$

$rootV \leftarrow Find(v)$

如果 $rootU = rootV$ ：

说明 u 和 v 已经连通

当前边 (u, v) 会形成环

否则：

合并两个集合

$fa[rootU] \leftarrow rootV$

函数 $Find(x)$ ：

1. 当 $fa[x] \neq x$ 时：

$x \leftarrow fa[x]$

2. 返回 x

(三) 并查集+路径压缩

为进一步提高并查集的查询效率，本实验在基础并查集的基础上引入路径压缩（Path Compression）优化策略。

路径压缩的核心思想是在执行查找操作时，将查找路径上的所有结点直接连接到根节点，从而降低树的高度。当再次访问这些结点时，可以直接到达根节点，减少重复遍历带来的开销。

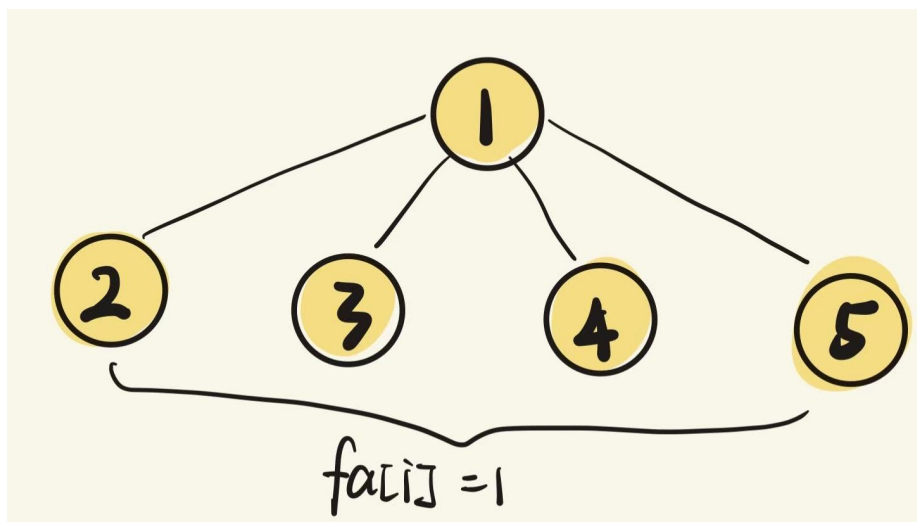


图 5 路径压缩优化示意图

在算法执行过程中，对于每条边 (u, v) ，仍然需要首先判断两个端点是否属于同一集合。若不属于同一集合，则执行合并操作；否则说明该边会形成环。与普通并查集相比，其主要区别在于查找函数采用了递归路径压缩方式，使每次查找结束后自动更新父节点信息。

引入路径压缩后，单次 find 的均摊复杂度约为 $O(\alpha(V))$ 。但本实验用该方法判断桥边时，仍然需要对每条边执行一次“删边后重建并查集”，因此总时间复杂度约为 $O(E^2 \alpha(V))$ ，空间复杂度为 $O(V)$ 。

路径压缩只能降低每次查找的常数开销，不能改变重复重建图带来的平方级总复杂度。

算法：并查集算法（路径压缩优化）

输入：图 $G(V, E)$

输出：判断每条边加入时是否形成环

1. 初始化父节点数组 fa

2. 对于每个顶点 i :

$fa[i] \leftarrow i$

3. 对于图中的每一条边 (u, v) :

$rootU \leftarrow \text{Find}(u)$

$rootV \leftarrow \text{Find}(v)$

如果 $rootU = rootV$:

说明 u 和 v 已经连通

当前边 (u, v) 会形成环

否则:

合并两个集合

$fa[rootU] \leftarrow rootV$

函数 $\text{Find}(x)$:

```

1. 如果  $fa[x] = x$ :
    返回  $x$ 
2. 否则:
     $fa[x] \leftarrow Find(fa[x])$ 
    返回  $fa[x]$ 

```

(四) 链表储存+并查集+生成森林

链表实现的并查集是一种用链式结构表示集合的并查集方法。与常见的树形并查集不同，链表实现中，每一个集合都被存储为一条链表，链表中的每个结点代表集合中的一个元素。为了方便管理集合，每个集合通常维护两个指针：**head** 指向链表的第一个结点，**tail** 指向链表的最后一个结点。执行 $FindSet(x)$ 操作时，只需要直接返回结点 x 中保存的 **head** 指针，就可以知道元素 x 所在的集合，因此查找操作的时间复杂度为 $O(1)$ 。

当执行 $Union(x, y)$ 操作时，首先分别通过 $FindSet(x)$ 和 $FindSet(y)$ 找到元素 x 和元素 y 所在的集合。如果两个元素已经属于同一个集合，则不需要进行合并操作。如果它们属于不同集合，则需要把两条链表连接起来。

若按集合大小合并链表，单次合并需要更新较小集合中的结点标记，代价为 $O(k)$ 。由于每个结点被并入更大集合的次数有限，建生成森林的总合并开销约为 $O(V \log V)$ ，扫描边的开销为 $O(E)$ 。

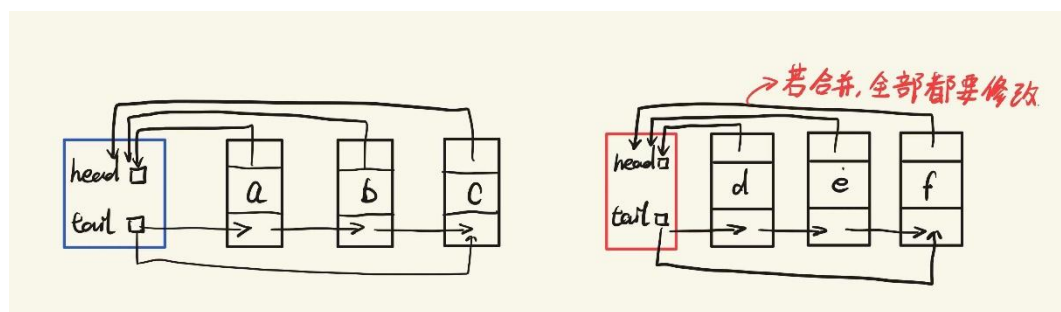


图 6 链表并查集合并示意图

需要说明的是，生成森林环覆盖并不是链表并查集独有的方法，普通数组并查集或路径压缩并查集也可以配合这一策略。本实验将它放在“链表+并查集+生成森林”中，只是因为集合部分采用链表表示。该方法的速度主要来自一次扫描建森林和非树边环覆盖，而不是链表本身。配合跳指针跳过已覆盖树边后，整体时间复杂度接近线性，约为 $O(E + V \log V)$ 。

链表并查集本身并不能直接判断某一条边是否为桥，它的作用是判断一条边的两个端点在当前生成森林中是否已经连通。算法利用“桥不属于任何环”的性质：如果一条边加入图中时，它的两个端点已经位于同一个连通集合中，则说明在当前生成森林中这两个端点之间已经存在一条路径，此时再加入该边会形成一个环。因此，这条边是非树边，非树边本身一定不是桥，同时它与生成森林中两端点之间的路径共同构成一个环，该路径上的所有树边也都不是桥。

伪代码如下：

```

MakeSet(x)
{
    create a new set S;
}

```

```
S.head = x;
S.tail = x;
x.next = NULL;
x.set = S;
}

FindSet(x)
{
    return x.set;
}

Union(x, y)
{
    Sx = FindSet(x);
    Sy = FindSet(y);

    if (Sx == Sy)
        return;

    Sx.tail.next = Sy.head;
    Sx.tail = Sy.tail;

    p = Sy.head;
    while (p != NULL)
    {
        p.set = Sx;
        p = p.next;
    }
}
```

(五) 可撤销并查集与线段树分治

可撤销并查集与线段树分治是一种用于离线处理动态连通性问题的方法。基准算法通常会枚举每一条边，每次删除一条边后重新使用 DFS 或 BFS 判断图的连通性，但这样会造成大量重复遍历，时间效率较低。为了避免重复计算可以尝试使用线段树分治和可撤销并查集进行优化。

设图中共有 m 条边，将第 i 次查询定义为“删除第 i 条边后图的连通状态”。这样，每一个查询时间点都对应一次删边操作。对于第 i 条边来说，它只在第 i 个时间点不存在，而在其他所有时间点都存在。因此，每条边都可以得到若干个存在时间区间。例如，第 i 条边在时间 i 被删除，所以它在区间 $[1, i-1]$ 和 $[i+1, m]$ 内存在。对于这些存在区间，可以使用线段树进行维护。线段树的叶子节点表示单个查询时间点，也就是一次删边操作。这样，在之后遍历线段树时，只需要在进入某个区间节点时将该节点保存的边加入并查集。

例如我们有四条边：对于边 e_1 来说，它除了时间 1 不存在，其他时间都存在（ e_1 存在于 $[2,4]$ ）

叶子节点/时间点：

时间 1：删除 e_1
时间 2：删除 e_2
时间 3：删除 e_3
时间 4：删除 e_4

线段树：

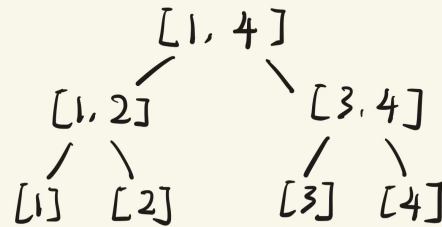


图 7 可撤销并查集时间区间示意图

图 8 线段树分治挂载示意图

如果某条边存在于 $[3,4]$ ，就挂到节点 $[3,4]$ 上。DFS 到 $[3,4]$ 时，把这条边加入并查集。这样搜索到叶子节点时，原先的边只用在父节点 $[3,4]$ 加入一次就好了。

在线段树上进行 DFS 遍历时，并查集的状态会随着遍历过程动态变化。进入某个线段树节点时，将该节点中保存的所有边加入可撤销并查集；当 DFS 到达某个叶子节点时，该叶子节点对应一个具体的删边查询，此时并查集中恰好保存了删除某一条边后仍然存在的所有边，因此可以直接通过并查集判断该边是否为桥；当 DFS 离开某个线段树节点时，需要撤销进入该节点后进行的所有合并操作。这个“可撤销的”操作主要依赖栈数据结构。

对于每一条边，由于它只在自身对应的删除时间点不存在，因此最多会被拆分成两个存在区间。每个存在区间插入线段树时，会被分解到 $O(\log m)$ 个线段树节点中，因此所有边在线段树上的总存储次数为 $O(m \log m)$ 。在线段树 DFS 过程中，每条被挂在线段树节点上的边都会被加入一次，并在回溯时撤销一次，因此总的合并和撤销次数也是 $O(m \log m)$ 。如果可撤销并查集采用按集合大小合并或按秩合并，单次 find 和 union 操作的复杂度可以控制在 $O(\log n)$ 以内，时间复杂度可以表示为 $O(m \log m \log n)$ 。该方法的伪代码如下所示：

Input: $G = (V, E)$, $\text{edge}[1 \dots m]$

Output: bridgeCount

$\text{origin} = \text{countConnectedComponents}(G)$

$\text{buildSegmentTree}(1, m)$

for $i = 1$ to m :

 if $i > 1$:

$\text{insert}(\text{edge}[i], 1, i - 1)$

 if $i < m$:

$\text{insert}(\text{edge}[i], i + 1, m)$

$\text{initRollbackUnionFind}()$

$\text{component} = |V|$

$\text{bridgeCount} = 0$

```
procedure solve(node):
    checkpoint = stack.size()

    for each edge (u, v) in node:
        ru = find(u)
        rv = find(v)

        if ru != rv:
            if size[ru] < size[rv]:
                swap(ru, rv)

            stack.push(ru, rv, size[ru], component)
            parent[rv] = ru
            size[ru] = size[ru] + size[rv]
            component = component - 1

    if node is leaf:
        if component > origin:
            bridgeCount = bridgeCount + 1
    else:
        solve(node.left)
        solve(node.right)

    while stack.size() > checkpoint:
        ru, rv, oldSize, oldComponent = stack.pop()
        parent[rv] = rv
        size[ru] = oldSize
        component = oldComponent

solve(root)

return bridgeCount
```

五、实验结果及分析：

1. 图 2 小规模例子正确性验证

将图 2 中的 16 个顶点按从上到下、从左到右的顺序编号为 0~15，并按报告中红色线段构造无向图。6 种方法输出的桥边集合完全一致，均为 (0, 1)、(2, 3)、(2, 6)、(6, 7)、(9, 10)、(12, 13)，说明各方法在小规模例子上能够正确识别桥边。

表 1：验证算法正确性

方法	桥边数	验证结论
蛮力法	6	与图 2 红色桥边一致
并查集	6	与图 2 红色桥边一致
并查集+路径压缩	6	与图 2 红色桥边一致
链表+并查集+生成森林	6	与图 2 红色桥边一致
可撤销并查集与线段树分治	6	与图 2 红色桥边一致
Tarjan	6	与图 2 红色桥边一致

2. mediumG 数据性能测试

本实验将其边表按无向图读取。由于规模较小，为降低计时误差，每种方法重复 10000 次后取平均单次用时，单位统一换算为 ms。

表 2：中规模下算法的效率

方法	桥边数	运行时间 (ms)	状态
蛮力法	0	0.207	OK
并查集	0	0.224	OK
并查集+路径压缩	0	0.227	OK
链表+并查集+生成森林	0	0.006	OK
可撤销并查集与线段树分治	0	0.012	OK
Tarjan	0	0.006	OK

3. largeG 数据性能测试

largeG.txt 包含 1000000 个顶点和 7586063 条边。蛮力法、朴素并查集、并查集+路径压缩均属于删边后重建连通性的口径，在该规模下预计运行超过 3 小时，因此按未完整运行记录；链表+并查集+生成森林、可撤销并查集与线段树分治、Tarjan 均实际运行并输出桥边数。

表 2：大规模下算法的效率

方法	桥边数	运行时间 (ms)	状态
蛮力法	-	>10800000	未完整运行 (3h++)
并查集	-	>10800000	未完整运行 (预计 3h+)
并查集+路径压缩	-	>10800000	未完整运行 (预计 3h+)

链表+并查集+生成森林	8	615.134	OK
可撤销并查集与线段树	8	5331.204	OK
分治			
Tarjan	8	1036.660	OK

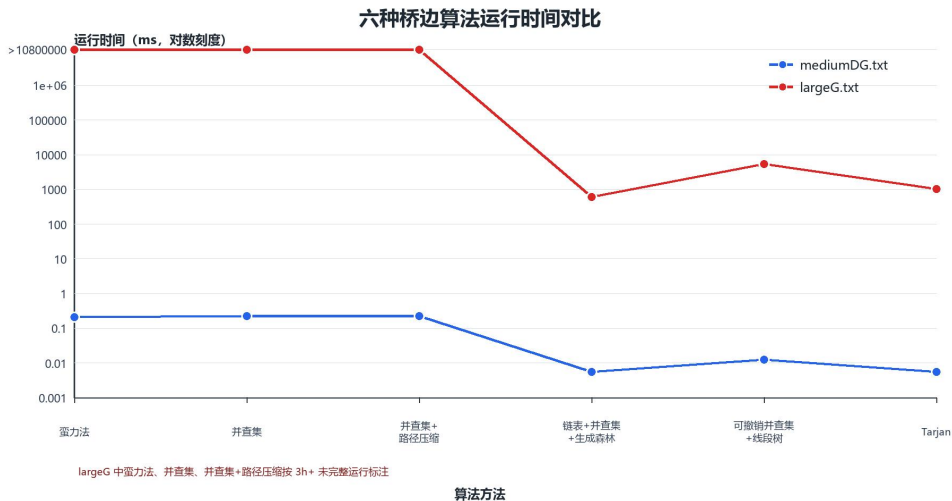


图 9 六种桥边算法运行时间曲线（纵轴为对数刻度，单位 ms）

六、实验结论：

图 2 小规模例子中，6 种方法输出的桥边完全一致，验证了实现的正确性。
mediumDG.txt 按无向图读取后没有桥边，6 种方法的桥边数均为 0。
largeG.txt 中，链表+并查集+生成森林、可撤销并查集与线段树分治和 Tarjan 均得到 8 条桥边；其中链表+并查集+生成森林用时 615.134ms，可撤销并查集与线段树分治用时 5331.204ms，Tarjan 用时 1036.660ms。
朴素并查集与路径压缩优化虽然查找操作更快，但仍然需要对每条边执行一次删边重建，因此在 largeG.txt 上不能与线性 Tarjan 直接比较。生成森林环覆盖不是链表独有的思想，普通并查集也可以用同样策略实现。本实现中链表+并查集+生成森林较快，主要是因为它只维护生成森林，并用跳指针跳过已覆盖树边，常数开销较小；这不表示链表结构本身优于 Tarjan。可撤销并查集与线段树分治通过离线分治和回滚减少重复建图，能够处理大规模数据，但复杂度仍高于 Tarjan。



指导教师批阅意见:

成绩评定:

指导教师签字:

2026 年 6 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。